

Building AI Agents with LangGraph

Assignment

Building a Customer Support Agent with LangGraph

Scenario

A technology firm, TechTrend Innovations, offers software solutions to customers globally. Their customer support staff handles many inquiries each day, including resolving software problems and asking for feature enhancements. The team finds it difficult to handle repetitive inquiries, monitor user history through interactions, and efficiently escalate complicated problems to human agents. The organization seeks an AI-driven customer support representative capable of managing frequent inquiries, keeping track of conversation records, and engaging human agents for intricate problems, enhancing response speed and customer contentment.

Problem Statement

Create a Customer Support Agent with LangGraph that helps TechTrend Innovations' clients by handling frequent questions (for instance, "How can I change my password?"), preserving conversation records between sessions, and transferring complicated matters to human representatives through a human-in-the-loop (HITL) process. The agent must utilize a systematic memory framework to retain user information, incorporate state management for immediate interactions, and enable long-term memory for monitoring user history. The solution should be implemented with a User Interface.

Objectives

- Design an AI agent that can handle customer support queries using LangGraph's state and memory management.
- Implement a memory schema to store short-term session data and long-term user history.
- Incorporate HITL to escalate complex queries to human agents for review.
- Ensure the agent is scalable, maintainable, and user-friendly for customer support tasks.

Tasks to Be Implemented

- **Define a Memory Schema**

Create AgentState with fields for short-term memory (e.g., messages) and long-term memory (e.g., user_history with past queries and resolutions) and include metadata like user_id and thread_id.

- **Implement State Management**
 - Use StateGraph to manage the agent's state during interactions.
 - Implement a state reducer to handle message updates (e.g., append new messages to messages).
- **Handle Short-Term and Long-Term Memory**
 - Use short-term memory to store the current session's messages.
 - Implement long-term memory using MemorySaver or an external store to persist user_history across sessions.
- **Process Customer Queries**
 - Create nodes for processing queries (e.g., an LLM node with a mock LLM to respond to common queries like "How do I reset my password?").
 - Add a tool node to fetch user history and provide personalized responses (e.g., "Last time, you asked about billing...").
- **Trim and Filter Messages**
 - Implement message trimming to manage context window size (e.g., keep only the last 5 messages in short-term memory).
 - Filter out irrelevant messages (e.g., greetings) to focus on actionable queries.
- **Configuration and Running**
 - Securely manage API keys using environment variables.
 - Allow local usage and support easy deployment to Community Cloud.
 - Document all steps in a clear README file.

Required Feature Suggestions:

- Multiple Schemas:
 - Use multiple schemas to separate user data (e.g., one schema for personal info, another for query history).
- External Memory Integration:
 - Integrate an external memory store (e.g., a database) to scale long-term memory.
- Enhanced UX:
 - Add user-friendly prompts in the Agent Chat UI for HITL (e.g., “Approve this resolution: Reset password link sent”).
- State Schema Validation:
 - Validate AgentState fields to ensure data consistency (e.g., user_id is a string, messages is a list).
- Frameworks for UI:
 - Frameworks can be used generally to enhance the interface (e.g., for custom styling or additional interactivity).

Note:

Make use of Virtual Environment to Avoid Version issues:

- Create a Virtual Environment:
 - Navigate to the project directory (e.g., D:\mod4\).
 - Run: `python -m venv venv`.
 - Activate the virtual environment:
 - Windows: `venv\Scripts\activate`
 - macOS/Linux: `source venv/bin/activate`
- Install Dependencies:
 - Create a `requirements.txt` with necessary packages (e.g., `langgraph==0.4.2`, `typing-extensions==4.12.2`, `langgraph-cli==0.2.8`).
 - Run: `pip install -r requirements.txt` to install dependencies in the virtual environment.